

When an agent needs to **buy** something.

Two open questions, the old answers to one of them, and what the payment world has been quietly building for the other.

YoungHwi Cho • Project7



Your agent hits something it **cannot do**. What now?

it needs

real web search

Not its training data. Today's pages.

it needs

a paid dataset

Behind a login and a subscription.

it needs

something rendered

A model it does not host.

Every one of these has a price on it. Someone is willing to sell. Nobody built the counter.

This is the whole subject. Everything after this is a way of answering it.

It breaks into **two questions**. They have very different answers.

question one

Does the agent have to learn every seller?

Every API has its own docs, SDK, request shape, error style. Do we teach the agent all of it, one at a time?

people have worked on this for a decade

question two

How does the agent actually pay?

Not "which card". An agent cannot open an account, accept terms, or clear a captcha. Someone has to.

this one is new, and it is moving fast

question one · does the agent learn every seller?

One API is fine. The **tenth one** is where you stop.

find it which one is even good

read the docs an hour, if they are good

install the sdk or write the client

learn the shape every one is different

auth, errors, retries every one is different

test it then test it again next month

and then

**Do it again.
And again.**

A day per API, on a good day. Nobody does this a hundred times, so agents end up with the five or six tools someone had time to wire by hand.

question one · the answer people already built

Put **one thing** in front of many things.

This idea is old and it works. Each generation did it a little better.

2015 →

API marketplaces

One account, one key, many APIs. RapidAPI still lists over 40,000.

one account

2019 →

Actor stores

Prebuilt jobs you rent by the run. Apify has more than 10,000.

pay per run

2024 →

MCP

Tools described in a way the agent can read by itself.

agent readable

2026 →

MCP gateways

Aggregators of the aggregators. Around 17 of them already.

one endpoint

Question one is, honestly, mostly solved.

RapidAPI Hub, Apify Store, MCP gateway landscape Q1 2026. Checked Sep 2026.

Every one of them still needs **a person with a card.**

what they solved

one interface

one place to look

one bill at the end

the agent can read it

what they did not

you still sign up

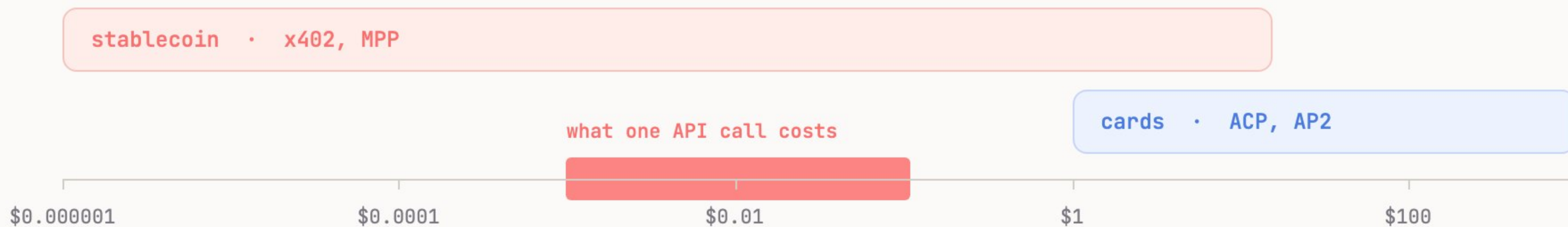
you still add a card

you still accept terms

the agent still cannot

The interface got solved. The counter never did. So the agent gets all the way to the checkout and hands the problem back to you.

Before picking a rail, look at **the size** of what an agent actually buys.



cards have a floor

A fixed fee per transaction. On \$20 it is nothing. On a \$0.002 call it is the whole thing.

stablecoins have a ceiling

No chargebacks, no disputes, no merchant who already takes you. Fine at a cent. Not fine at a thousand dollars.

Four standards. They are not fighting over **the same money**.

machines buying from machines

APIs, compute, data

Per call. Fractions of a cent. Nobody watching.



x402

Coinbase, now Linux Foundation

T MPP

Stripe and Tempo

agents buying from shops

Goods, checkouts, plans

Dollars. A cardholder behind it. Shops that exist.



ACP

Stripe and OpenAI



AP2

Google

One of them appears twice. That is the interesting part.

Stripe did not pick a side. They built **both**, then a bridge.



Same challenge, either rail. A merchant reconciles both in one place, so they never have to care which one the agent used.

Two worlds, and they built **the same thing**.

S shared payment token

for one seller
up to one amount
until it expires
kept by Stripe

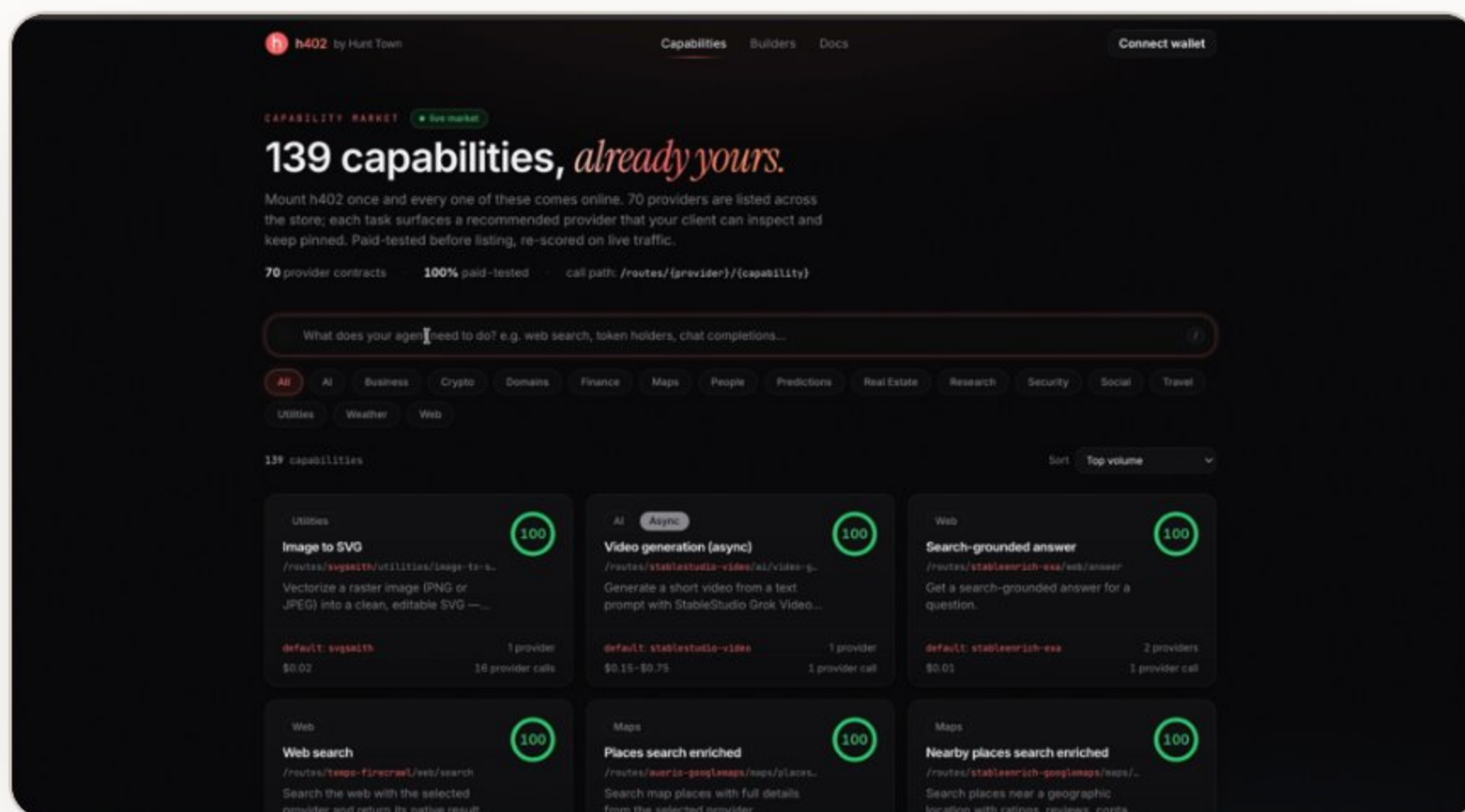
eip-3009 authorization

for one address
up to one amount
until validBefore
kept by the agent

A one time permission to move a set amount to a set counterparty. The difference is not the idea. It is *who holds it*.

so where does that leave us

Now the agent can pay. It still does not know **who to pay.**



every listing tells you

the name

the price

the endpoints

none of them tells you

does it work

how often does it fail

is it slower than the other one

So it picks the first one, or the cheapest, and finds out the hard way.

what we built

Mount it once. Then just ask for the thing.

```
/routes/{provider}/{category}/{action} // the hundredth one is shaped like the first
```

~140

capabilities

across 16 categories

71

provider contracts

competing on the same paths

100%

paid-tested

we bought every route first

Base

settles here

USDC over x402, per call

No keys. No accounts. No SDK per seller. The price shows up before anything is charged.

Live figures move. Check `h402-test.hunt.town` on the day.

the part that is actually work

A listing is a claim. A payment that settled is **a fact.**

01

We buy it

Nothing gets listed until we have paid for it ourselves.

02

We watch it

Success rate, latency, price.
From real calls, not from docs.

03

We rank it

Providers on the same task,
scored against each other.

04

We pin one

The agent gets an answer, not
a menu.

and every later call feeds it back

A list can be copied in a week. A year of paid calls cannot.

back to the four standards

The agent should not have to know **which one won.**

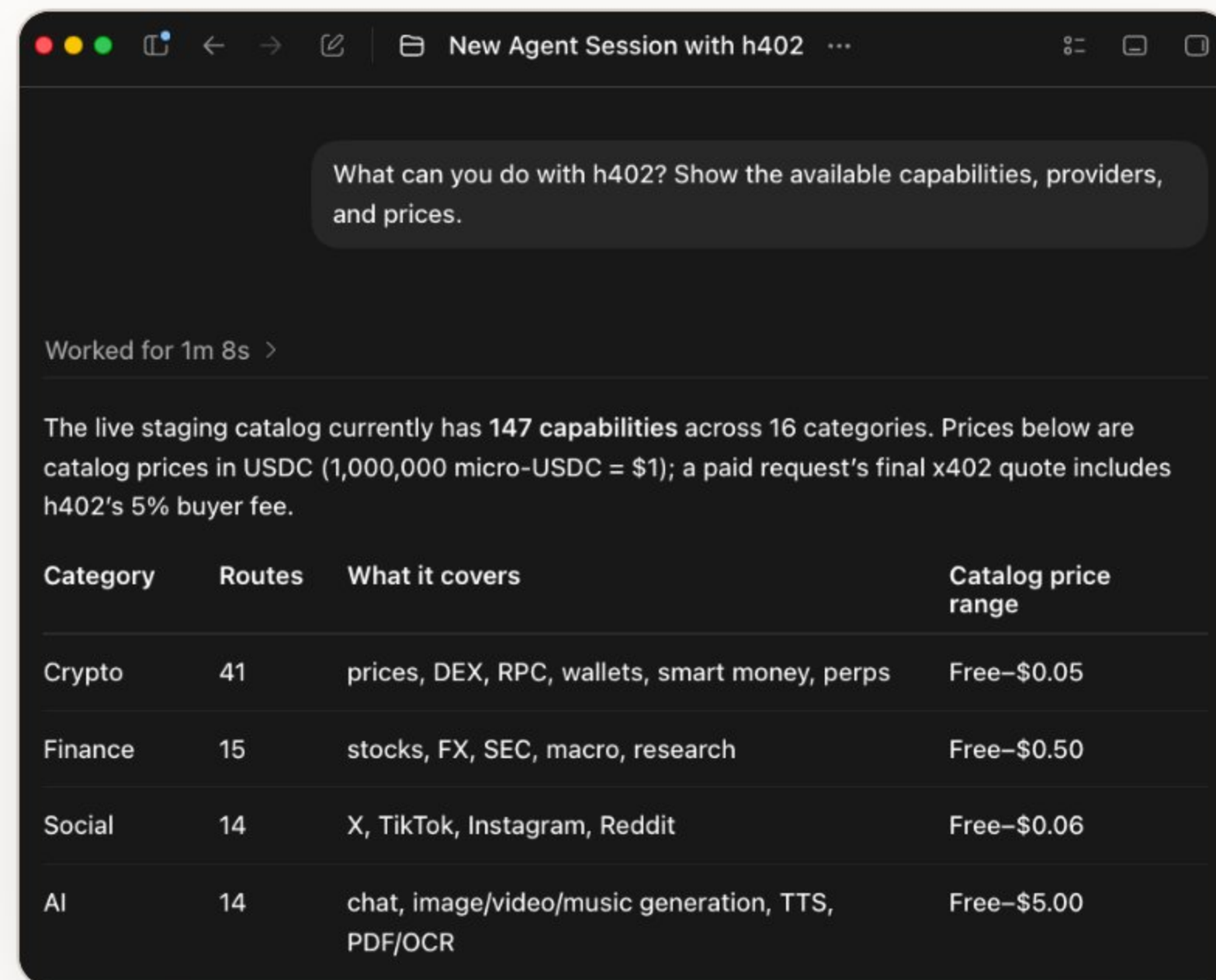


A browser does not ask you which version of TLS you would prefer. This should feel the same.

live

A session that has never seen this. One link.

Watch what it decides to buy, and what it shows you before
it spends anything.



Stop asking what your agent knows. Ask **what it can buy.**

Building agents?

Your next design problem is a budget and a spend cap, not a prompt. Nobody has good defaults for that yet.

Have an API?

Agents are customers who cannot sign up for anything. A middleware and a day is the whole lift.

Picking a protocol?

Do not. They are not competing for the same money. Sit behind an abstraction and let the price decide.